CSC3002
Assignment 1: Data Types, Functions, Operators, Strings, and Streams

Due: October 8, 2026, 23:59

Learning objectives

In this assignment you will define and implement a small fixed-capacity string type. You will practise namespaces, structs, constructors, assignment, references, function overloading, operator overloading, and stream input/output.

Task

The release package contains `string.h`, `string.cpp`, and `test_assignment1.cpp`.

1. Complete every function and operator declared in `string.h`, placing your implementation in `string.cpp`.
2. Keep all declarations, names, signatures, and the namespace `mystring` unchanged.
3. Implement the operations using character-by-character loops. Do not use C or C++ string-library functions such as `strlen`, `strcpy`, `strcat`, `strcmp`, `std::string`, or `std::getline`.
4. Use `test_assignment1.cpp` to test your implementation before submission.

The type `mystring::str` stores a null-terminated character sequence in `data`. Its maximum content length is `maxlen`; the extra array element is reserved for the terminating null character. Your implementation must work when the instructor changes `maxlen` to any positive value.

Required behaviour

- Copying and concatenation preserve the character order and always write a final null character.
- If an input or concatenated result is longer than `maxlen`, store the first `maxlen` characters and discard the rest. This rule also applies to stream extraction.
- `str_len` excludes the null terminator; `str_clear` produces an empty string.
- `upper` and `lower` change only English alphabetic characters and return a new `str`; the input is not modified.
- Comparison operators use lexicographical order, like dictionary order based on character values.

- `operator+` returns a new string and does not modify its operands. `operator+=` appends to its left operand and returns that operand by reference.
- `operator«` writes the stored characters without adding a newline. `operator»` reads one whitespace-delimited word.

What to submit

Submit your completed `string.cpp` to Blackboard by the deadline.

Run the test program

Use the supplied `test_assignment1.cpp` to test your implementation. It reports each test and a score out of 10. In the VS Code integrated terminal, compile the program with:

```
g++ -std=c++17 -Wall -Wextra -pedantic string.cpp test_assignment1.cpp -o
    test_assignment1
```

On macOS, run:

```
./test_assignment1
```

On Windows termial, run:

```
.\test_assignment1.exe
```

Academic integrity

Submit work that you understand and can explain. **We may randomly select students with different grades and ask them during tutorials to explain their code and answer TA questions. Full-AI-use and plagiarism detection will be conducted.** Detected inappropriate AI use or copied work will affect your grade.